

EXPT NO: 7	GENERATE SYNTHETIC IMAGES USING A GENERATIVE ADVERSARIAL NETWORK (GAN)
DATE: 5/10/26	

PRE-LAB QUESTIONS (PROVIDE BRIEF ANSWERS TO THE FOLLOWING QUESTIONS)

1. What is a Generative Adversarial Network (GAN)?

A **Generative Adversarial Network (GAN)** is a deep learning model used to generate new data that resembles real data. It consists of two neural networks: a **Generator** and a **Discriminator**. The Generator creates synthetic data, while the Discriminator evaluates whether the data is real or generated. Both networks are trained simultaneously in competition with each other. GANs are commonly used for image generation, image enhancement, data augmentation, and creating realistic synthetic images.

2. What are the roles of the Generator and Discriminator in a GAN?

A GAN consists of two main components:

- **Generator:** Creates synthetic images from random noise. Its goal is to generate images that are realistic enough to fool the Discriminator.
- **Discriminator:** Classifies input images as either **real** or **synthetic**. Its goal is to correctly distinguish generated images from real images.

The Generator improves its image generation based on the Discriminator's feedback, while the Discriminator becomes better at detecting fake images.

3. Why are GANs used for synthetic image generation?

GANs are used for synthetic image generation because they can learn the underlying patterns and features of real images and generate new, realistic-looking images. They are useful when large amounts of real training data are unavailable or difficult to collect. GANs can generate diverse images while preserving important visual characteristics. Applications include **face generation, medical imaging, image-to-image translation, super-resolution, data augmentation, and realistic scene generation**.

4. Difference between Real Images and Synthetic Images

Feature	Real Images	Synthetic Images
Source	Captured from the real world	Generated using computational models
Creation	Camera or imaging device	AI/model-based generation
Authenticity	Represents actual objects/scenes	Artificially created
Variability	Naturally occurring	Controlled by the generation process
Examples	Photograph of a person	GAN-generated person's face

5. Explain the concept of adversarial training in GANs.

Adversarial training is the process where the Generator and Discriminator compete against each other during GAN training. The Generator tries to create realistic synthetic images that can fool the Discriminator. The Discriminator tries to correctly identify whether an image is real or generated. This competition continuously improves both networks. Eventually, the Generator learns to produce highly realistic images that are difficult for the Discriminator to distinguish from real images.

AIM

To implement a Generative Adversarial Network (GAN) for generating synthetic images by training a Generator and Discriminator on an image dataset and evaluating the quality of the generated images.

SOFTWARE REQUIREMENTS

- Python
- TensorFlow / Keras
- Google Colab
- GitHub
- NumPy
- Matplotlib
- Pandas

OBJECTIVES

Students will be able to:

- Understand the architecture and working of Generative Adversarial Networks.
- Understand the roles of the Generator and Discriminator.
- Prepare and preprocess an image dataset for GAN training.
- Build a Generator network for synthetic image generation.
- Build a Discriminator network for distinguishing real and generated images.
- Train a GAN using adversarial learning.
- Generate synthetic images using the trained Generator.
- Visualize and analyze the quality of generated images.

PROCEDURE

PART A: DATASET PREPARATION AND IMAGE PREPROCESSING

Perform the following tasks:

- Load an image dataset such as the MNIST handwritten digit dataset.
- Explore the dataset and examine sample images.
- Check the number of images and image dimensions.
- Display sample images from the dataset.
- Normalize the pixel values to the range $[-1, 1]$.
- Reshape the images into the required input format.
- Prepare the dataset for batch-wise training.
- Create a data pipeline using appropriate TensorFlow/Keras utilities.

PART B: IMPLEMENTING THE GENERATOR

Perform the following tasks:

- Define a random noise vector as the input to the Generator.
- Create the Generator using appropriate neural network layers.
- Use Dense and convolutional/transposed convolutional layers to generate images.
- Apply appropriate activation functions.
- Generate an image from a random noise vector.
- Display the generated image before GAN training.
- Display the Generator model architecture and summary.

PART C: IMPLEMENTING THE DISCRIMINATOR AND GAN MODEL

Perform the following tasks:

- Create a Discriminator network to classify images as real or generated.

- Add convolutional layers to extract image features.
- Add appropriate activation functions.
- Add a final output layer for binary classification.
- Compile the Discriminator using:
 - Appropriate optimizer
 - Binary cross-entropy loss
- Combine the Generator and Discriminator to form the GAN.
- Display the Generator and Discriminator model summaries.

PART D: GAN TRAINING AND SYNTHETIC IMAGE GENERATION

Perform the following tasks:

- Initialize the Generator with random noise.
- Train the Discriminator using:
 - Real images from the dataset.
 - Synthetic images generated by the Generator.
- Train the Generator through the combined GAN model.
- Record:
 - Generator loss
 - Discriminator loss
- Repeat the training process for the specified number of epochs.
- Generate synthetic images at regular training intervals.
- Visualize generated images at different stages of training.
- Compare the generated images with real images.
- Analyze the quality and diversity of the generated images.
- Save the final generated images.

EXPECTED OUTCOME

Students will understand the architecture and working of Generative Adversarial Networks (GANs) and the interaction between the Generator and Discriminator during adversarial training. Students will learn to preprocess image data, build and train GAN models, and generate synthetic images. They will evaluate the generated images by visually comparing them with real images and analyzing the improvement in image quality during training.

POST-LAB QUESTIONS (PROVIDE BRIEF ANSWERS TO THE FOLLOWING QUESTIONS)

1. What is the role of the Generator in a GAN?

The **Generator** is a neural network that creates synthetic data, such as images, from random noise. Its main objective is to produce realistic data that resembles the real training data. During training, the Generator receives feedback from the Discriminator and learns to improve its outputs. Over time, it becomes better at generating realistic images that are difficult for the Discriminator to distinguish from real images.

2. What is the role of the Discriminator in a GAN?

The **Discriminator** is a neural network that distinguishes between **real data** from the training dataset and **synthetic data** produced by the Generator. It outputs a probability indicating whether an input is real or fake. Its objective is to correctly classify both types of data. The feedback provided by the Discriminator helps the Generator improve the realism and quality of its generated images.

3. Why is random noise used as input to the Generator?

Random noise provides the Generator with a starting point from which it can create different synthetic samples. By using different random noise vectors, the Generator can produce diverse images rather than repeatedly generating the same image. The noise is transformed through the Generator's neural network into meaningful features and eventually into a complete image. Thus, random noise helps GANs generate varied and realistic outputs.

4. What happens when the Discriminator becomes too strong during GAN training?

If the **Discriminator becomes too strong**, it can easily distinguish real images from generated images. As a result, the Generator receives very weak or uninformative gradients and struggles to improve. This can make GAN training unstable and slow down Generator learning. The generated images may remain poor in quality. Therefore, maintaining a reasonable balance between the Generator and Discriminator is important for stable and effective GAN training.

5. What is meant by mode collapse in GANs?

Mode collapse is a common problem in GAN training where the Generator produces very similar or identical outputs instead of generating diverse samples. The Generator discovers a limited type of output that successfully fools the Discriminator and repeatedly generates it. Although these images may look realistic, they lack diversity and fail to represent the different patterns present in the training dataset.

6. How does the Generator improve the quality of generated images during training?

The Generator improves through **feedback from the Discriminator**. Initially, it produces poor-quality images. The Discriminator identifies them as fake and provides a learning signal through its loss. The Generator uses **backpropagation and gradient-based optimization** to update its weights. Through repeated training iterations, it learns important patterns, features, textures, and structures from real data, gradually producing more realistic and higher-quality images.

7. Mention one real-world application of GANs.

One important real-world application of GANs is **image generation**. GANs can generate realistic human faces that do not belong to real people. They are also used in **medical image synthesis, image super-resolution, data augmentation, image-to-image translation, fashion design, and game development**. For example, GAN-generated images can increase the amount of training data available for machine learning models when collecting real images is expensive or difficult.

8. Difference between GAN Training and Conventional Neural Network Training

Feature	GAN Training	Conventional Neural Network Training
Networks	Two networks: Generator and Discriminator	Usually one main network
Objective	Networks compete with each other	Minimize prediction error
Training	Adversarial	Direct optimization
Feedback	Discriminator guides Generator	Loss function guides model
Main Goal	Generate realistic data	Make accurate predictions

SUGGESTIONS FOR DATASETS

- MNIST Handwritten Digits – <https://www.tensorflow.org/datasets/catalog/mnist>
- Fashion-MNIST – https://www.tensorflow.org/datasets/catalog/fashion_mnist
- CIFAR-10 – <https://www.tensorflow.org/datasets/catalog/cifar10>
- CelebA – https://www.tensorflow.org/datasets/catalog/celeb_a
- Oxford-IIIT Pet Dataset – https://www.tensorflow.org/datasets/catalog/oxford_iiit_pet

Screenshots

The screenshot shows a Jupyter Notebook titled "GAN_MNIST_Synthetic_Image_Generation.ipynb" by Pradeep Manikandan D 24BAD088. The notebook is in the "PART A: DATASET PREPARATION AND IMAGE PREPROCESSING" section. The text explains that the MNIST dataset is loaded, explored, visualized, normalized to the range [-1, 1], reshaped, and prepared as a TensorFlow data pipeline for batch-wise training.

```
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
print("TensorFlow version:", tf.__version__)
print("Num GPUs Available:", len(tf.config.list_physical_devices('GPU')))
```

Output: TensorFlow version: 2.21.0

```
# Load the MNIST dataset
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()

print("Training images shape:", x_train.shape)
print("Training labels shape:", y_train.shape)
print("Testing images shape:", x_test.shape)
print("Testing labels shape:", y_test.shape)
print("Image dimensions:", x_train.shape[1:])
print("Number of training images:", len(x_train))
print("Number of testing images:", len(x_test))
```

Output: Training images shape: (60000, 28, 28), Training labels shape: (60000,), Testing images shape: (10000, 28, 28), Testing labels shape: (10000,), Image dimensions: (28, 28), Number of training images: 60000, Number of testing images: 10000

The screenshot shows the same Jupyter Notebook, now displaying a grid of 20 handwritten digits from the MNIST dataset, each with its corresponding label. The digits are arranged in a 4x5 grid. The labels are: 5, 0, 4, 1, 9, 2, 1, 3, 1, 4, 3, 5, 3, 6, 1, 7, 2, 8, 6, 9.

```
# Check pixel-value range before preprocessing
print("Minimum pixel value:", x_train.min())
print("Maximum pixel value:", x_train.max())

# Normalize pixel values from [0, 255] to [-1, 1]
x_train = x_train.astype("float32")
x_train = (x_train - 127.5) / 127.5
```

Output: Minimum pixel value: 0, Maximum pixel value: 255

```
# After preprocessing
print("After preprocessing:")
print("Shape:", x_train.shape)
print("Minimum pixel value:", x_train.min())
print("Maximum pixel value:", x_train.max())
```

Output: After preprocessing: Shape: (60000, 28, 28, 1), Minimum pixel value: -1.0, Maximum pixel value: 1.0

```
# Create TensorFlow data pipeline
BATCH_SIZE = 128
BUFFER_SIZE = 60000

train_dataset = (
    tf.data.Dataset.from_tensor_slices(x_train)
    .shuffle(BUFFER_SIZE)
    .batch(BATCH_SIZE, drop_remainder=True)
    .prefetch(tf.data.AUTOTUNE)
)
```

Output: Batch size: 128

GAN_MNIST_Synthetic_Image_Generation.ipynb

Generate

+ Code

+ Markdown

Run All

Restart

Python 3.10.0

```

for batch in train_dataset.take(1):
    print("One batch shape:", batch.shape)

```

[5]

✓ 0.5s

Python

Batch size: 128

One batch shape: (128, 28, 28, 1)

Generate

+ Code

+ Markdown

PART B: IMPLEMENTING THE GENERATOR

The Generator receives a random noise vector and transforms it into a 28×28×1 image. Dense and transposed-convolution layers are used to progressively transform the latent vector into an image.

```

# Define Generator parameters
LATENT_DIM = 100

def build_generator():
    model = keras.Sequential(name="Generator")

```

[6]

✓ 0.3s

Python

GAN_MNIST_Synthetic_Image_Generation.ipynb

Generate

+ Code

+ Markdown

Run All

Restart

Python 3.10.0

```

# Convert noise vector into a small feature map
model.add(layers.Dense(7 * 7 * 128, use_bias=False))
model.add(layers.BatchNormalization())
model.add(layers.LeakyReLU())

model.add(layers.Reshape((7, 7, 128)))

# Upsample: 7x7 -> 14x14
model.add(layers.Conv2DTranspose(
    64, kernel_size=5, strides=2, padding="same", use_bias=False
))
model.add(layers.BatchNormalization())
model.add(layers.LeakyReLU())

# Upsample: 14x14 -> 28x28
model.add(layers.Conv2DTranspose(
    1, kernel_size=5, strides=2, padding="same",
    use_bias=False, activation="tanh"
))

return model

generator = build_generator()
generator.summary()

```

[6]

✓ 0.3s

Python

Model: "Generator"

GAN_MNIST_Synthetic_Image_Generation.ipynb

Generate

+ Code

+ Markdown

Run All

Restart

Python 3.10.0

Model: "Generator"

Layer (type)	Output Shape	Params
dense (Dense)	(None, 6272)	6272
batch_normalization (BatchNormalization)	(None, 6272)	256
leaky_re_lu (LeakyReLU)	(None, 6272)	0
reshape (Reshape)	(None, 7, 7, 128)	0
conv2d_transpose (Conv2DTranspose)	(None, 14, 14, 64)	2048
batch_normalization_1 (BatchNormalization)	(None, 14, 14, 64)	256
leaky_re_lu_1 (LeakyReLU)	(None, 14, 14, 64)	0
conv2d_transpose_1 (Conv2DTranspose)	(None, 28, 28, 1)	1

Total params: 858,944 (3.28 MB)

GAN_MNIST_Synthetic_Image_Generation.ipynb

Generate

+ Code

+ Markdown

Run All

Restart

Python 3.10.0

Trainable params: 846,272 (3.23 MB)

Non-trainable params: 12,672 (49.50 KB)

```

# Generate an image before GAN training
random_noise = tf.random.normal([1, LATENT_DIM])
generated_image = generator(random_noise, training=False)

print("Generated image shape:", generated_image.shape)

```

[7]

✓ 0.1s

Python

Generated image shape: (1, 28, 28, 1)

```

# Display generated image before training
plt.figure(figsize=(3, 3))
plt.imshow(generated_image[0, :, :, 0], cmap="gray")
plt.title("Generated Image Before Training")
plt.axis("off")
plt.show()

```

[8]

✓ 0.3s

Python

Pradeep Manikandan D 24BAD088 -----DL Lab Expt 7

GAN_MNIST_Synthetic_Image_Generation.ipynb

Generate

Code

Markdown

Run All

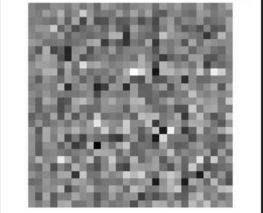
Restart

Python 3.10.0

[#] ✓ 0.3s

Python

Generated Image Before Training



PART C: IMPLEMENTING THE DISCRIMINATOR AND GAN MODEL

```
def build_discriminator():
    model = keras.Sequential(name="Discriminator")

    model.add(layers.Input(shape=(28, 28, 1)))

    model.add(layers.Conv2D(
        64, kernel_size=5, strides=2, padding="same"
    ))
    model.add(layers.LeakyReLU())
    model.add(layers.Dropout(0.3))

    model.add(layers.Conv2D(
        128, kernel_size=5, strides=2, padding="same"
    ))
    model.add(layers.LeakyReLU())
    model.add(layers.Dropout(0.3))

    model.add(layers.Flatten())
    model.add(layers.Dense(1, activation="sigmoid"))

    return model

discriminator = build_discriminator()
discriminator.summary()
```

[9] ✓ 0.2s Python

GAN_MNIST_Synthetic_Image_Generation.ipynb

Generate

Code

Markdown

Run All

Restart

Python 3.10.0

Layer (type)

Output Shape

Par

conv2d (Conv2D)	(None, 14, 14, 64)	
leaky_re_lu_2 (LeakyReLU)	(None, 14, 14, 64)	
dropout (Dropout)	(None, 14, 14, 64)	
conv2d_1 (Conv2D)	(None, 7, 7, 128)	28
leaky_re_lu_3 (LeakyReLU)	(None, 7, 7, 128)	
dropout_1 (Dropout)	(None, 7, 7, 128)	
flatten (Flatten)	(None, 6272)	
dense_1 (Dense)	(None, 1)	

Total params: 212,865 (831.50 KB)

Trainable params: 212,865 (831.50 KB)

Non-trainable params: 0 (0.00 B)

```
# Define loss function and optimizer
cross_entropy = keras.losses.BinaryCrossentropy(from_logits=False)
generator_optimizer = keras.optimizers.Adam(
    learning_rate=0.0002, beta_1=0.5
)
discriminator_optimizer = keras.optimizers.Adam(
    learning_rate=0.0002, beta_1=0.5
)
print("Loss function: Binary Cross-Entropy")
print("Optimizer: Adam")
```

[10] ✓ 0.0s Python

```
# Test the Discriminator with real and generated images
real_batch = next(iter(train_dataset))
fake_batch = generator(
    tf.random.normal([BATCH_SIZE, LATENT_DIM]),
    training=False
)

real_predictions = discriminator(real_batch, training=False)
fake_predictions = discriminator(fake_batch, training=False)
```

[11] ✓ 0.5s Python

The image displays a Jupyter Notebook titled "GAN_MNIST_Synthetic_Image_Generation.ipynb" with Python 3.10.0. The notebook is divided into two main sections: "GAN Architecture" and "Training configuration".

GAN Architecture:

```
# Test the Discriminator with real and generated images
real_batch = next(iter(train_dataset))
fake_batch = generator(
    tf.random.normal([BATCH_SIZE, LATENT_DIM]),
    training=False
)

real_predictions = discriminator(real_batch, training=False)
fake_predictions = discriminator(fake_batch, training=False)

print("Real prediction shape:", real_predictions.shape)
print("Fake prediction shape:", fake_predictions.shape)
print("Example real prediction:", float(real_predictions[0][0]))
print("Example fake prediction:", float(fake_predictions[0][0]))
```

Output:

```
Real prediction shape: (128, 1)
Fake prediction shape: (128, 1)
Example real prediction: 0.5238209366798401
Example fake prediction: 0.5008296966552734
```

Training configuration:

```
EPOCHS = 30
NUM_EXAMPLES_TO_GENERATE = 16
# Fixed seed allows comparison of image quality during training
seed = tf.random.normal([NUM_EXAMPLES_TO_GENERATE, LATENT_DIM])
generator_losses = []
discriminator_losses = []
generated_snapshots = []
print("Epochs:", EPOCHS)
print("Batch size:", BATCH_SIZE)
print("Latent dimension:", LATENT_DIM)
```

Output:

```
Epochs: 30
Batch size: 128
Latent dimension: 100
```

def train_step(images):

```
@tf.function
def train_step(images):
    # Generate random noise
    noise = tf.random.normal([BATCH_SIZE, LATENT_DIM])

    with tf.GradientTape() as gen_tape, tf.GradientTape() as disc_t:
        # Generate synthetic images
        generated_images = generator(noise, training=True)

        # Discriminator predictions
        real_output = discriminator(images, training=True)
        fake_output = discriminator(generated_images, training=True)

        # Calculate Generator loss
        gen_loss = cross_entropy(
            tf.ones_like(fake_output), fake_output
        )

        # Calculate Discriminator loss
        real_loss = cross_entropy(
            tf.ones_like(real_output), real_output
        )
        fake_loss = cross_entropy(
            tf.zeros_like(fake_output), fake_output
        )
        disc_loss = real_loss + fake_loss

        # Calculate gradients
        gradients_of_generator = gen_tape.gradient(
            gen_loss, generator.trainable_variables
        )
        gradients_of_discriminator = disc_t.gradient(
            disc_loss, discriminator.trainable_variables
        )

        # Update model weights
        generator_optimizer.apply_gradients(
            zip(gradients_of_generator, generator.trainable_variables)
        )
        discriminator_optimizer.apply_gradients(
            zip(gradients_of_discriminator, discriminator.trainable_variables)
        )

    return gen_loss, disc_loss
```

GAN_MNIST_Synthetic_Image_Generation.ipynb

```

for epoch in range(1, EPOCHS + 1):
    epoch_gen_losses = []
    epoch_disc_losses = []
    for image_batch in train_dataset:
        gen_loss, disc_loss = train_step(image_batch)
        epoch_gen_losses.append(float(gen_loss))
        epoch_disc_losses.append(float(disc_loss))
    avg_gen_loss = np.mean(epoch_gen_losses)
    avg_disc_loss = np.mean(epoch_disc_losses)
    generator_losses.append(avg_gen_loss)
    discriminator_losses.append(avg_disc_loss)
    if epoch == 1 or epoch % 5 == 0 or epoch == EPOCHS:
        snapshot = generator(seed, training=False).numpy()
        generated_snapshots.append((epoch, snapshot))
    print(
        f"Epoch {epoch:02d}/{EPOCHS} | "
        f"Generator Loss: {avg_gen_loss:.4f} | "
        f"Discriminator Loss: {avg_disc_loss:.4f}"
    )

```

```

Epoch 01/30 | Generator Loss: 0.7898 | Discriminator Loss: 1.2125
Epoch 02/30 | Generator Loss: 0.7794 | Discriminator Loss: 1.3085
Epoch 03/30 | Generator Loss: 0.8536 | Discriminator Loss: 1.2329
Epoch 04/30 | Generator Loss: 0.9610 | Discriminator Loss: 1.1522
Epoch 05/30 | Generator Loss: 0.9820 | Discriminator Loss: 1.1735

```

GAN_MNIST_Synthetic_Image_Generation.ipynb

```

# Display generated images from different training stages
fig, axes = plt.subplots(
    len(generated_snapshots),
    16,
    figsize=(16, 2 * len(generated_snapshots))
)

if len(generated_snapshots) == 1:
    axes = np.expand_dims(axes, axis=0)

for row, (epoch, images) in enumerate(generated_snapshots):
    for col in range(16):
        axes[row, col].imshow(images[col, :, :], cmap="gray")
        axes[row, col].axis("off")

    axes[row, 0].set_ylabel(
        f"Epoch {epoch}",
        rotations=90,
        fontsize=12
    )

plt.suptitle("GAN Generated Images During Training", fontsize=16)
plt.tight_layout()
plt.show()

```

```

Epoch 27/30 | Generator Loss: 0.9329 | Discriminator Loss: 1.2261
Epoch 28/30 | Generator Loss: 0.9354 | Discriminator Loss: 1.2272
Epoch 29/30 | Generator Loss: 0.9317 | Discriminator Loss: 1.2282
Epoch 30/30 | Generator Loss: 0.9389 | Discriminator Loss: 1.2240

```

Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output

Visualize Generated Images at Different Training Stages

```

# Display generated images from different training stages
fig, axes = plt.subplots(
    len(generated_snapshots),
    16,
    figsize=(16, 2 * len(generated_snapshots))
)

if len(generated_snapshots) == 1:
    axes = np.expand_dims(axes, axis=0)

for row, (epoch, images) in enumerate(generated_snapshots):
    for col in range(16):
        axes[row, col].imshow(images[col, :, :], cmap="gray")
        axes[row, col].axis("off")

    axes[row, 0].set_ylabel(
        f"Epoch {epoch}",
        rotations=90,
        fontsize=12
    )

plt.suptitle("GAN Generated Images During Training", fontsize=16)
plt.tight_layout()
plt.show()

```

GAN_MNIST_Synthetic_Image_Generation.ipynb

Generate

Code

Markdown

Run All

Restart

Python 3.10.0

```

# Plot Generator and Discriminator losses
plt.figure(figsize=(10, 5))

plt.plot(
    range(1, EPOCHS + 1),
    generator_losses,
    label="Generator Loss"
)

plt.plot(
    range(1, EPOCHS + 1),
    discriminator_losses,
    label="Discriminator Loss"
)

plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("GAN Training Loss")
plt.legend()
plt.grid(True)
plt.show()

```

[16]

✓ 0.1s

Python

Compare Real Images with Generated Images

GAN_MNIST_Synthetic_Image_Generation.ipynb

Generate

Code

Markdown

Run All

Restart

Python 3.10.0

```

# Display real MNIST images
real_images = x_train[:16]
# Generate synthetic images
final_noise = tf.random.normal([16, LATENT_DIM])
final_generated_images = generator(final_noise, training=False).numpy()
plt.figure(figsize=(16, 4))
for i in range(16):
    plt.subplot(2, 16, i + 1)
    plt.imshow(real_images[i, :, :, 0], cmap="gray")
    plt.title("Real")
    plt.axis("off")
    plt.subplot(2, 16, i + 1 + 16)
    plt.imshow(final_generated_images[i, :, :, 0], cmap="gray")
    plt.title("Fake")
    plt.axis("off")
plt.tight_layout()
plt.show()

```

[17]

✓ 1.1s

Python

Save Final Generated Images

The generated images are saved as PNG files in a folder named `generated_images`. A ZIP file is also created for convenient download from Google Colab.

```

# Create output directory
output_dir = "generated_images"
os.makedirs(output_dir, exist_ok=True)

# Generate final images
num_final_images = 100
final_noise = tf.random.normal([num_final_images, LATENT_DIM])
final_images = generator(final_noise, training=False).numpy()

```

[18]

✓ 0.2s

Python

The screenshot shows a Jupyter Notebook with the following code and output:

```
# Convert [-1, 1] back to [0, 255]
final_images_uint8 = (
    ((final_images + 1.0) * 127.5)
    .clip(0, 255)
    .astype(np.uint8)
)

# Save individual images
for i in range(num_final_images):
    image_path = os.path.join(output_dir, f"generated_{i+1:03d}.png")
    keras.utils.save_img(
        image_path,
        final_images_uint8[i],
        scale=False
    )

print(f"Saved {num_final_images} generated images to: {output_dir}")
```

Output: Saved 100 generated images to: generated_images

```
fig = plt.figure(figsize=(10, 10))
for i in range(100):
    plt.subplot(10, 10, i + 1)
    plt.imshow(final_images_uint8[i, :, :], cmap="gray")
    plt.axis("off")
plt.suptitle("Final GAN Generated MNIST Images", fontsize=16)
plt.tight_layout()
contact_sheet_path = "generated_images/final_contact_sheet.png"
plt.savefig(contact_sheet_path, dpi=150, bbox_inches="tight")
plt.show()
print("Contact sheet saved to:", contact_sheet_path)
```

Output: Final GAN Generated MNIST Images

Output: Contact sheet saved to: generated_images/final_contact_sheet.png

```
# Create ZIP file containing generated images
import shutil

zip_path = shutil.make_archive(
    "GAN_generated_MNIST_images",
    "zip",
    output_dir
)

print("ZIP file created:", zip_path)
```

Output: ZIP file created: GAN_generated_MNIST_images.zip

Model Parameter Summary

The following cells display the total number of trainable parameters in the Generator and Discriminator.

```
# Display parameter counts
generator_params = generator.count_params()
discriminator_params = discriminator.count_params()

summary_df = pd.DataFrame({
    "Model": ["Generator", "Discriminator"],
    "Total Parameters": [generator_params, discriminator_params]
})

display(summary_df)
```

Model	Total Parameters
0 Generator	858944
1 Discriminator	212865

GitHub: <https://github.com/Pradeep-Manikandan-1-D/24BAD088> DEEP LEARNING LAB
ASSESSMENT

Description	Max Marks	Marks Awarded
Pre Lab Exercise	5	
In Lab Exercise	10	
Post Lab Exercise	5	
Viva	10	
Total	30	
Faculty Signature		